

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Макеев И.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.10.26

Москва, 2026

Постановка задачи

Вариант 20.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процессы child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки.

Программа должна быть написана на языке Си, при этом необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс (копию вызывающего процесса).
- `int pipe(int pipefd[2])`; – создает неименованный канал: `pipefd[0]` – конец для чтения, `pipefd[1]` – конец для записи.
- `int dup2(int oldfd, int newfd)`; – переназначает файловый дескриптор; с его помощью стандартный ввод потомка заменяется концом канала, а стандартный вывод – файлом.
- `int open(const char *pathname, int flags, mode_t mode)`; – открывает (создает) файл для записи.
- `int close(int fd)`; – закрывает файловый дескриптор.
- `int execl(const char *path, const char *arg, ...)`; – заменяет образ памяти процесса новой программой (вариант семейства `exec`).
- `pid_t waitpid(pid_t pid, int *status, int options)`; – ожидает завершения дочернего процесса и получает его статус.
- `ssize_t write(int fd, const void *buf, size_t count)`; – записывает данные в файловый дескриптор (в канал).
- `void exit(int status)`; – завершает выполнение процесса и возвращает статус.
- `sighandler_t signal(int signum, sighandler_t handler)`; – используется, чтобы игнорировать сигнал `SIGPIPE`.

В работе реализованы две программы: `parent.c` (родительский процесс) и `child.c` (дочерний процесс). Родитель запускается пользователем, а дочерняя программа запускается им через `exec1`, поэтому процессы представлены разными исполняемыми файлами.

- Родитель читает от пользователя два имени файла (для child1 и child2) и создает два канала: `pipe1` и `pipe2`.
- Для каждого потомка вызывается `fork`. В потомке открывается свой файл на запись, затем с помощью `dup2` стандартный ввод заменяется читающим концом своего канала, а стандартный вывод – файлом. После этого потомок закрывает все лишние дескрипторы и вызывает `execl`, заменяя свой образ программой `child`.

- Родитель закрывает читающие концы каналов и в цикле читает строки произвольной длины с помощью `getline`. Если длина строки больше 10 символов, строка записывается в `pipe2`, иначе в `pipe1`.
- Дочерняя программа читает строки из стандартного ввода, инвертирует каждую из них и выводит в стандартный вывод, который перенаправлен в файл.
- Когда пользователь завершает ввод (`Ctrl+D`), родитель закрывает записывающие концы каналов. Потомки получают признак конца файла, дочитывают данные и завершаются. Родитель ожидает их завершения через `waitpid` и проверяет коды завершения.

Важная деталь: в каждом потомке закрываются все концы обоих каналов, кроме использованного. Канал считается открытым на запись, пока открыт хотя бы один его записывающий конец, поэтому лишний открытый дескриптор не позволил бы потомку получить конец файла, и он не завершился бы. Обработка ошибок: проверяются результаты `pipe`, `fork`, `open`, `dup2`, `execl`, `write` и `waitpid`, сообщения выводятся через `perror`, при неуспешном завершении потомка родитель возвращает ненулевой код. Сигнал `SIGPIPE` игнорируется, чтобы при аварийном завершении потомка запись в канал давала ошибку, а не убивала родителя.

Код программы

parent.c

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

pid_t start_child(const char *file, int in[2], int other[2]) {
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return -1;
    }
    if (pid == 0) {
        int out = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (out == -1) {
            perror("open");
            exit(1);
        }
        if (dup2(in[0], STDIN_FILENO) == -1 || dup2(out, STDOUT_FILENO) == -1) {
            perror("dup2");
            exit(1);
        }
        close(out);
        close(in[0]);
        close(in[1]);
        close(other[0]);
        close(other[1]);

        execl("./child", "child", NULL);
        perror("execl");
        exit(1);
    }
    return pid;
}
```

```

int main(void) {
    signal(SIGPIPE, SIG_IGN);

    char *line = NULL;
    size_t cap = 0;

    printf("Файл для child1: ");
    fflush(stdout);
    if (getline(&line, &cap, stdin) == -1) return 1;
    line[strcspn(line, "\n")] = '\0';
    if (line[0] == '\0') {
        fprintf(stderr, "Имя файла не введено\n");
        return 1;
    }
    char *file1 = strdup(line);

    printf("Файл для child2: ");
    fflush(stdout);
    if (getline(&line, &cap, stdin) == -1) return 1;
    line[strcspn(line, "\n")] = '\0';
    if (line[0] == '\0') {
        fprintf(stderr, "Имя файла не введено\n");
        return 1;
    }
    char *file2 = strdup(line);

    if (file1 == NULL || file2 == NULL) {
        perror("strdup");
        return 1;
    }

    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = start_child(file1, pipe1, pipe2);
    pid_t pid2 = -1;
    if (pid1 != -1) pid2 = start_child(file2, pipe2, pipe1);

    close(pipe1[0]);
    close(pipe2[0]);

    int result = 0;

    if (pid1 == -1 || pid2 == -1) {
        result = 1;
    } else {
        printf("Вводите строки (Ctrl+D - конец):\n");
        while (getline(&line, &cap, stdin) != -1) {
            size_t n = strcspn(line, "\n");
            line[n] = '\n';

            // длиннее 10 символов - во второй канал, иначе в первый
            int fd = (n > 10) ? pipe2[1] : pipe1[1];
            if (write(fd, line, n + 1) == -1) {
                perror("write");
                result = 1;
                break;
            }
        }
    }
}

```

```

close(pipe1[1]);
close(pipe2[1]);

int status;
if (pid1 > 0) {
    waitpid(pid1, &status, 0);
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) result = 1;
}
if (pid2 > 0) {
    waitpid(pid2, &status, 0);
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) result = 1;
}

free(file1);
free(file2);
free(line);
return result;
}

```

child.c

```

#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *line = NULL;
    size_t cap = 0;

    while (getline(&line, &cap, stdin) != -1) {
        size_t n = strcspn(line, "\n");
        line[n] = '\0';

        for (size_t i = 0; i < n / 2; i++) {
            char t = line[i];
            line[i] = line[n - 1 - i];
            line[n - 1 - i] = t;
        }

        if (printf("%s\n", line) < 0 || fflush(stdout) == EOF) {
            perror("write");
            free(line);
            return 1;
        }
    }

    free(line);
    return 0;
}

```

Протокол работы программы

Тест 1. Компиляция и обычная работа:

```

$ gcc -Wall -Wextra -o child child.c
$ gcc -Wall -Wextra -o parent parent.c
$ ./parent
Файл для child1: out1.txt
Файл для child2: out2.txt
Вводите строки (Ctrl+D - конец):
hello
abc
this is a long line
0123456789

```

```
01234567890
[Ctrl+D]
$ cat out1.txt
olleh
cba
9876543210
$ cat out2.txt
enil gnol a si siht
09876543210
```

Строки hello, abc и 0123456789 (не длиннее 10 символов) попали в pipe1 и в файл child1, а строки длиннее 10 символов – в pipe2 и в файл child2. Граничные случаи (10 и 11 символов) обработаны верно.

Тест 2. Ошибка открытия файла:

```
$ ./parent
Файл для child1: /nonexistent/x.txt
Файл для child2: out2.txt
Вводите строки (Ctrl+D - конец):
[Ctrl+D]
open: No such file or directory
```

Вывод

В ходе лабораторной работы были изучены создание процессов через fork и замена их образа через exec1, передача данных между процессами через неименованные каналы и переназначение стандартных потоков ввода-вывода с помощью dup2. Реализована программа, в которой родитель распределяет строки между двумя дочерними процессами по длине, а потомки инвертируют их и записывают в файлы. Основное внимание при отладке потребовалось уделить закрытию неиспользуемых концов каналов: пока остается открытым лишний записывающий конец, потомок не получает конец файла и не завершается. Также была реализована обработка системных ошибок.